

Grammarmcetamol — Implementation Phases & Delivery Roadmap

Document Version: 1.0

Last Updated: 2026-08-01

Methodology: Vertical slicing per Epic, dependency-first, risk-reduction priority

Sprint Cadence: 2-week sprints (recommended)

How to Read This Document

Each phase is a **vertical slice** — it delivers a working, testable increment across backend, frontend, and infrastructure. We do not build “all backends first”; instead, we ship end-to-end features that create immediate value.

Legend	Meaning
 Hard Dependency	Cannot start until predecessor is Done
 Soft Dependency	Can start in parallel, but needs stubs/mocks
 Parallel Safe	No blockers, can run concurrently
 Milestone	Demo-ready checkpoint

Phase 0: Foundation & DevEx (Sprints 1–2)

Goal: Every developer can `git clone`, run `docker-compose up`, and see a healthy stack.

0.1 Infrastructure & Tooling

Task	Owner	Output
Provision AWS dev account (EKS, RDS, S3, ElastiCache, RabbitMQ)	DevOps	Terraform modules
Set up monorepo structure (<code>services/</code> , <code>apps/admin</code> , <code>apps/student</code> , <code>shared/</code>)	Platform	Repo scaffold
CI/CD pipeline per service (GitHub Actions → Docker Build → ECR → Helm deploy)	DevOps	<code>.github/workflows/</code>
Local development stack (<code>docker-compose.dev.yml</code> with PostgreSQL, MongoDB, Redis, RabbitMQ, MinIO)	Platform	<code>docker/</code> folder
API Gateway skeleton (Spring Cloud Gateway) with routing config	Backend	<code>gateway-service/</code>
Observability baseline (Prometheus + Grafana + Loki local stacks)	DevOps	<code>observability/</code>

0.2 Shared Libraries

Task	Output
Common DTOs / event schemas (Avro or JSON Schema)	shared/events/
Base Docker images (JRE 21 slim, Node 20 Alpine)	shared/docker/
Internal npm package: @grammarcetamol/ui (design tokens)	shared/ui/

0.3 Database Baseline

Task	Output
Run Phase 0 migrations (create databases, users, extensions)	migrations/V0__baseline.sql
MongoDB cluster init + replica set config	migrations/mongo/001_baseline.js

✔ Phase 0 Exit Criteria

- docker-compose up brings up 100% of local stack in <3 min.
- Any service push to main triggers build, test, and deploy to dev EKS.
- Grafana shows green health for all services.
- Developer onboarding doc is written.

Phase 1: Identity, Access & User Management (Sprints 3–5)

Goal: A user can register, verify email, log in, and have a profile. Admins can create moderator accounts.

- **Hard Dependency:** Phase 0 (infrastructure must exist)

★ **Milestone:** Authentication is production-ready; RBAC is functional.

1.1 Backend Services

Auth Service (Java / Spring Boot)

User Story	Acceptance Criteria
US-STU-001: Account Registration	Email/password + Google OAuth; bcrypt hashing (cost 12); welcome email queued
US-STU-002: Email Verification	24h token expiry; resend capability; status transitions to active
US-STU-003: Secure Login	JWT access (15 min) + refresh (7 days); 5-attempt logout; silent refresh flow
US-GUEST-006: Seamless Auth Redirect	returnUrl persisted through OAuth and email-verification flows

Database: auth_db — run V1__auth_initial_schema.sql

Events Published: user.created, user.verified, user.login, user.logout, user.locked

User Service (Node.js / NestJS)

User Story	Acceptance Criteria
US-STU-013: Profile Management	CRUD profile; avatar upload (signed URL to S3); timezone auto-detect
US-ADM-012: Moderator Management	Super Admin creates Moderator; granular permission matrix stored
US-ADM-013: Student Management	Admin directory; suspend/activate with reason; view enrollment + transaction history

Database: `user_db` — run `V1__user_initial_schema.sql`

Events Published: `profile.updated`, `role.assigned`, `user.suspended`

Events Consumed: `user.created` (to auto-create profile), `user.verified`

1.2 Frontends

Student Frontend (Next.js)

Page	Features
<code>/login</code>	Email + password form; Google OAuth button; “Forgot Password” link
<code>/register</code>	Validation (8 chars, mixed case, number); confirm password; T&Cs checkbox
<code>/forgot-password</code>	Email input; success state; rate-limited (1 req / 60s)
<code>/verify-email</code>	Token validation; redirect to dashboard on success
<code>/profile</code>	Tabs: Profile, Account, Notifications, Privacy; avatar cropper

Admin Frontend (Next.js)

Page	Features
<code>/login</code>	Admin-only login; 2FA prompt if enforced
<code>/users</code>	DataTable of all users; filter by role/status; bulk actions
<code>/users/create</code>	Create Moderator wizard; permission matrix checkbox grid
<code>/users/[id]</code>	Profile read-only + activity log + permission editor
<code>/roles</code>	View role definitions; edit Moderator permissions

1.3 Cross-Cutting

Task	Detail
RabbitMQ exchanges	<code>user.exchange</code> created; bindings for <code>user.*</code> routing keys

Table 9 – continued

Task	Detail
Redis	Session store TTL 30m; JWT blacklist TTL 1d
API Gateway	Route config: <code>/api/auth/**</code> → Auth Service; <code>/api/users/**</code> → User Service; AuthFilter active
Middleware	<code>middleware.ts</code> route guards on both frontends

✅ Phase 1 Exit Criteria

- End-to-end test: Guest → Register → Verify Email → Login → View Profile → Logout.
- Security audit: Password hashing, JWT expiry, refresh rotation, rate limiting.
- Admin can create a Moderator and assign `course:edit` but not `financial:view`.
- Load test: `/api/auth/login` handles 500 req/s with $p95 < 200\text{ms}$.

Phase 2: Course Content & Discovery (Sprints 6–9)

Goal: Guests can browse courses; Admins can create courses via a wizard; content is searchable.

- 🔴 **Hard Dependency:** Phase 1 (admins must be authenticated to create content)
- 🟡 **Soft Dependency:** Media Service can be stubbed (accept file, return mock URL)
- ★ **Milestone:** Public course catalog is live; first course can be created end-to-end.

2.1 Backend Services

Course Service (Java / Spring Boot)

User Story	Acceptance Criteria
US-GUEST-002: Browse Course Catalog	Paginated, filterable (category, difficulty, price, rating), full-text search
US-GUEST-003: View Course Details	Curriculum accordion, instructor card, reviews summary, pricing card
US-ADM-006: Guided Course Creation	5-step wizard; auto-save draft; validation per step; publish atomic guard
US-ADM-009: Course Editing & Versioning	Edit published course → new version snapshot; rollback capability; notify enrolled students on new lessons
US-ADM-010: Safe Course Deletion	Block delete if enrollments > 0 ; archive instead; hard delete only if zero activity

Database: `course_db` — run `V1__course_initial_schema.sql`

Events Published: `course.created`, `course.published`, `course.updated`, `course.archived`, `course.versioned`

Events Consumed: `user.created` (for instructor linkage), `enrollment.completed` (for completion stats)

Upload Service (Node.js / NestJS)

User Story	Acceptance Criteria
US-ADM-007: Resumable Chunked Upload	5MB chunks; SHA-256 checksum; 3 retries with exponential backoff; session recovery after browser crash

Database: upload_db — run V1__upload_initial_schema.sql

Storage: MinIO (dev) / S3 (prod); presigned PUT URLs for direct chunk upload

Events Published: upload.session.started, upload.chunk.completed, upload.file.completed, upload.failed

Media Service (Node.js / NestJS — MongoDB)

Task	Detail
Transcoding pipeline	On <code>upload.file.completed</code> , trigger <code>ffprobe</code> metadata extraction + HLS transcoding to 720p/1080p
Thumbnail generation	Auto-extract from first frame; allow custom poster upload
CDN delivery	CloudFront distribution in front of S3/MinIO

Database: media_db — run 001_media_initial.js

Events Consumed: upload.file.completed

Events Published: media.transcoding.started, media.transcoding.completed, media.ready

2.2 Frontends

Student Frontend

Page	Features
/ (Landing)	Hero, 7 service cards, featured course carousel, testimonials, FAQ accordion
/courses	Sidebar filters (category, difficulty, price, rating, duration); sort dropdown; CourseCard grid; infinite scroll
/courses/[slug]	Promo video thumbnail; curriculum accordion with lock icons; instructor bio; sticky price card; related courses

Admin Frontend

Page	Features
/courses	DataTable: thumbnail + title + instructor + status badge + price + students + rating; bulk actions; export CSV

Table 14 – continued

Page	Features
<code>/courses/create</code>	Step wizard: 1) Info (RichTextEditor, ImageUploader), 2) Pricing, 3) Structure (drag-drop ModuleManager), 4) Upload (FileUploader with chunk grid), 5) Review & Publish
<code>/courses/[id]</code>	Tabbed: Overview (metrics), Edit (change tracking), Content (lesson tree), Students (enrollment table), Analytics (charts), Versions (restore)
<code>/courses/[id]/upload</code>	Upload Manager: queue table, per-chunk status grid, session recovery banner, global controls

2.3 Cross-Cutting

Task	Detail
Search index	PostgreSQL <code>tsvector</code> on <code>courses.title/description</code> ; future: Elasticsearch
Redis caching	Course detail cache TTL 5m; catalog filter cache TTL 2m
Object Storage	Bucket policies: public-read for processed media, private for raw uploads

✅ Phase 2 Exit Criteria

- Admin creates a 3-module course with 5 lessons, uploads a 500MB video, publishes it.
- Guest visits `/courses`, filters by “Beginner”, clicks course, sees curriculum.
- Upload survives browser close + reopen with 100% resume accuracy.
- Video plays via HLS with adaptive bitrate switching.

Phase 3: Enrollment, Payments & Learning Loop (Sprints 10–14)

Goal: Students can buy/enroll in courses, watch videos with progress persistence, and receive certificates.

- 🔴 **Hard Dependency:** Phase 2 (courses must exist to enroll in)
- 🔴 **Hard Dependency:** Phase 1 (auth required for enrollment)
- ★ **Milestone:** First paid enrollment completes; student resumes video at exact timestamp.

3.1 Backend Services

Enrollment Service (Java / Spring Boot)

User Story	Acceptance Criteria
US-STU-006: Free Course Enrollment	Instant enrollment; idempotent (re-click is no-op); appears in “My Courses” immediately

Table 16 – continued

User Story	Acceptance Criteria
US-STU-005: Resume Learning	GET /progress/{courseId} returns last position per lesson; PATCH every 5s debounced
US-STU-008: Interactive Learning Interface	Lesson completion toggle; prerequisite gating (can't open Lesson 3 if Lesson 2 incomplete)
US-ADM-005: Student Engagement Insights	At-risk flagging (<20% progress after 14 days); completion rate aggregation

Database: enrollment_db — run V1__enrollment_initial_schema.sql

Events Published: enrollment.created, enrollment.completed, lesson.progress.updated

Events Consumed: course.published, payment.completed, user.created

Payment Service (Node.js / NestJS)

User Story	Acceptance Criteria
US-STU-007: Course Purchase & Checkout	Branded checkout; Stripe/Paystack/Flutterwave integration; PCI-compliant tokenization; idempotency keys
US-ADM-004: Revenue Analytics	Transaction ledger; refund workflow with approval gate; invoice generation

Database: payment_db — run V1__payment_initial_schema.sql

Events Published: payment.intent.created, payment.completed, payment.failed, refund.requested, refund.completed

Events Consumed: enrollment.created (for invoice generation)

Review Service (Java / Spring Boot)

User Story	Acceptance Criteria
US-STU-012: Leave Course Reviews	50% completion gate; 1–5 star + text; edit within 7 days; admin moderation queue

Database: review_db — run V1__review_initial_schema.sql

Events Published: review.submitted, review.approved, review.moderated

Events Consumed: enrollment.completed

3.2 Frontends

Student Frontend

Page	Features
/checkout/[courseId]	Order summary (course thumbnail, price breakdown); payment method selector; “Pay” button with loading state; success/failure states
/dashboard	Welcome banner; “Continue Learning” card with resume button; My Courses tabs; upcoming live classes; notifications; recommended courses
/my-courses	Grid of enrolled courses with progress bars; filter by status
/my-courses/[courseId]	3-pane learning interface: Left (lesson sidebar with progress), Center (video player + notes + nav), Right (instructor, downloads, discussion, bookmarks)
/my-courses/[courseId] (Mobile)	Bottom sheet lesson drawer; tabs for Notes/Discussion/Downloads; fullscreen video rotation

Admin Frontend

Page	Features
/revenue	Summary cards (lifetime, monthly, weekly); line chart toggles; donut charts by category; best-sellers horizontal bar
/transactions	DataTable: ID, date, student, course, amount, method, status; refund action with modal
/reviews	Kanban-style pipeline: Pending → Approved → Flagged; moderation actions
/students	Directory with advanced filters; profile drill-down (activity timeline, enrollments, progress, transactions)
/students/[id]	Avatar header; tabs: Activity, Enrollments, Progress, Transactions, Notes

3.3 Cross-Cutting

Task	Detail
Payment webhooks	Stripe <code>payment_intent.succeeded</code> → idempotent handler → publish <code>payment.completed</code>
Idempotency	Idempotency-Key header on all POST/PUT endpoints; Redis TTL 24h
Video player	Custom HTML5 + hls.js wrapper; keyboard shortcuts; progress sync every 5s; resume on return
Signed URLs	Resource downloads expire after 15 min; CloudFront signed cookies for video segments

✔ **Phase 3 Exit Criteria**

- Student buys a \$49 course; payment webhook fires; enrollment created in <2s.
- Student watches 12:34 of Lesson 2; closes tab; reopens → resumes at 12:34.
- Admin issues a refund; transaction ledger shows debit; student loses access (configurable grace period).
- Revenue dashboard shows the sale within 5 seconds of webhook receipt.

Phase 4: Live Classes & Notifications (Sprints 15–17)

Goal: Students see upcoming live classes, join on time, and receive timely notifications.

- **Hard Dependency:** Phase 1 (auth), Phase 3 (enrollment concept for class registration)

● **Soft Dependency:** Video conferencing can be external link (Zoom) initially, embedded SDK later

★ **Milestone:** First live class is scheduled, students register, and join via portal.

4.1 Backend Services

Live Class Service (Node.js / NestJS — MongoDB)

User Story	Acceptance Criteria
US-STU-009: Live Classroom Participation	Schedule class; capacity limit; register/join flow; 15-min early join gate; reminder emails at 24h/1h/15min
US-ADM-011: Live Class Scheduling	Admin scheduler with instructor conflict detection; recurring classes (future); calendar ICS generation

Database: `liveclass_db` — run `001_liveclass_initial.js`

Events Published: `liveclass.scheduled`, `liveclass.started`, `liveclass.ended`, `liveclass.reminder`

Events Consumed: `user.created` (for auto-enrollment if bundled)

Notification Service (Node.js / NestJS)

User Story	Acceptance Criteria
US-STU-014: Notification Center	Grouped by category; unread badge; mark-as-read; click-through navigation
US-NOTIF-001: Announcement Publishing	Target by segment; schedule future publish; expire; high-priority triggers email
US-NOTIF-002: Admin Alert Configuration	Configurable channels per event type; quiet hours; digest mode

Database: `notification_db` — run `V1_notification_initial_schema.sql`

Channels: In-app (SSE), Email (SendGrid/SES), SMS (future)

Events Consumed: All domain events (`payment.*`, `enrollment.*`, `course.*`, `liveclass.*`, `review.*`)

4.2 Frontends

Student Frontend

Page	Features
<code>/live-classes</code>	Tabs: Upcoming / Past; card with countdown; capacity indicator; Register/Join button state machine
<code>/live-classes/[id]/join</code>	Pre-join screen (5 min before); countdown; tech check; embedded Jitsi/Zoom or external link launcher
<code>/notifications</code>	Panel with category filters; unread dot; infinite scroll; mark-all-read

Admin Frontend

Page	Features
<code>/live-classes</code>	Calendar view (month/week/day) + list toggle; drag-to-reschedule; conflict warning toast
<code>/live-classes/create</code>	Form: title, instructor dropdown (with availability), datetime picker, capacity, price, meeting link
<code>/announcements</code>	Table: title, target, priority, status; create form with RichTextEditor; “Send Test” button
<code>/announcements/create</code>	Target audience selector (all, specific courses, segments); schedule picker; recipient count preview

4.3 Cross-Cutting

Task	Detail
SSE stream	<code>/api/notifications/stream</code> for real-time admin activity feed + student notifications
Email templates	Transactional templates in <code>notification_db</code> ; MJML for responsive email
Calendar ICS	<code>.ics</code> attachment on live class registration confirmation
Reminder cron	Distributed cron (Kubernetes CronJob) publishes <code>liveclass.reminder</code> events

✅ Phase 4 Exit Criteria

- Admin schedules a class for tomorrow; 3 students register; all receive reminder emails.
- Student clicks “Join” 10 min before start → pre-join screen → launches meeting at T-5min.
- Admin publishes a high-priority announcement → targeted students see in-app notification + email within 60s.
- Activity feed on admin dashboard shows the registration event in real time.

Phase 5: Service Requests, Support & Business Operations (Sprints 18–20)

Goal: Corporate clients can request services; support tickets flow through a pipeline; admins have full operational visibility.

- 🔴 **Hard Dependency:** Phase 1 (auth), Phase 4 (notifications for ticket updates)
- 🟡 **Soft Dependency:** Analytics can use pre-aggregated SQL initially
- ★ **Milestone:** First service request is submitted, reviewed by admin, and closed with satisfaction survey.

5.1 Backend Services

Service Request Service (Node.js / NestJS — MongoDB)

User Story	Acceptance Criteria
US-STU-011: Service Request Submission	7 flagship services catalog; multi-step request form; reference number; status tracking
US-ADM-014: Service Request Management	Kanban pipeline; assign to admin; internal notes; threaded communication log; status change emails

Database: request_db — run 001_servicerequest_initial.js

Events Published: servicerequest.created, servicerequest.status_changed

Events Consumed: user.created (for requester profile enrichment)

Admin Service (Java / Spring Boot)

User Story	Acceptance Criteria
US-ADM-001: Operational Dashboard	Metric cards (students, revenue, courses, tickets, requests); clickable drill-downs
US-ADM-002: Quick Actions Panel	One-click buttons: Create Course, Schedule Class, Publish Announcement, etc.
US-ADM-003: Real-Time Activity Feed	SSE-driven; filter by event type; 10s polling fallback
US-ADM-015: Support Ticket Handling	Priority + SLA tracking; threaded replies; canned responses; satisfaction survey on resolve

Database: admin_db — run V1__admin_initial_schema.sql

Audit: Every admin action logged to audit_logs (partitioned monthly)

Events Consumed: All domain events for dashboard + activity feed

Analytics Service (Node.js / NestJS — MongoDB)

User Story	Acceptance Criteria
US-ADM-004: Revenue Analytics	Time-series aggregation pipeline; daily/weekly/monthly rollups
US-ADM-005: Student Engagement Insights	At-risk scoring; popular search keywords; cohort retention

Database: analytics_db — run 001_analytics_initial.js

Pattern: Event sourcing from RabbitMQ → events collection → nightly MapReduce → daily_metrics

Events Consumed: All domain events

5.2 Frontends

Student Frontend

Page	Features
/services	7 service cards with illustrations; click to detail
/services/[slug]	Benefits, pricing, duration; “Request Service” CTA
/services/request	Multi-section form (organization, needs, dates, budget); confirmation screen with reference #

Admin Frontend

Page	Features
/ (Dashboard)	Metric cards row → Quick Actions → Revenue chart + Student insights + Activity feed + Upcoming classes + Pending requests
/service-requests	Kanban board (Received → Under Review → Responded → Scheduled → Completed); drag-drop; card detail sidebar
/service-requests/[id]	Requester info; service details; internal notes (threaded); communication timeline; status change + assignee dropdown
/support/tickets	Split-pane: ticket list left (filter tabs, search, preview cards), detail right
/support/tickets/[id]	Message thread (student left, admin right, system center); RichTextEditor reply; canned responses; “Send & Resolve”
/analytics/overview	Full-width charts; date range picker; export CSV
/analytics/students	Retention funnel; at-risk table; search keyword cloud
/settings/general	Platform name, logo upload, contact info
/settings/payments	Gateway config (masked API keys); webhook URL display; tax settings
/settings/security	Password policy, 2FA toggle, session timeout, login limits, IP whitelist
/settings/integrations	Connected services grid (Google, Zoom, Cloudinary); webhook management
/logs/audit	Immutable table: timestamp, admin, action, target, IP; filter + export CSV
/logs/system	Auto-refresh (30s); log level filter; stack trace expander; JSON export

✔ Phase 5 Exit Criteria

- Corporate visitor submits a training request → admin receives in-app + email alert → moves to “Under Review” → sends response → status changes to “Responded”.
- Student opens support ticket → admin replies within SLA → resolves → satisfaction email sent.
- Admin dashboard loads all metrics in <1s from pre-aggregated cache.
- Audit log shows every admin action with immutable timestamp + IP.

Phase 6: Performance, Security & Production Hardening (Sprints 21–23)

Goal: Platform is production-grade: fast, secure, observable, and compliant.

6.1 Performance

Task	Target
CDN + Edge caching	CloudFront caches static assets; API responses cached at edge for public endpoints
Database query optimization	Add missing indexes; EXPLAIN ANALYZE on slow queries; materialized views for dashboards
Video optimization	Per-title encoding; thumbnail WebP/AVIF; lazy load below fold
Bundle optimization	Route splitting; dynamic imports for charts, editor, video player; tree shaking

6.2 Security

Task	Standard
Penetration testing	OWASP Top 10 mitigation verified by external audit
DDoS protection	Cloudflare WAF + rate limiting; ALB anomaly detection
Secrets rotation	AWS Secrets Manager; 90-day rotation policy
Data encryption	RDS at-rest (AES-256); S3 SSE-KMS; TLS 1.3 in transit
Compliance	GDPR data export/delete; PCI DSS SAQ-A for payments

6.3 Observability

Task	Tool
Distributed tracing	Jaeger spans across all services
Log correlation	Trace ID injected into all logs; Loki logQL dashboards

Table 34 – continued

Task	Tool
Alerting	PagerDuty integration for 5xx errors, payment failures, disk >80%
SLOs	99.9% availability; p95 API latency <200ms; error rate <0.1%

6.4 Disaster Recovery

Task	RTO / RPO
Database backups	Daily snapshots; point-in-time recovery (PITR) 35 days
Cross-region replication	S3 bucket replication to secondary region
Runbook documentation	Incident response playbooks for each service

✅ Phase 6 Exit Criteria

- Lighthouse Performance score 90 on all public pages.
- Security audit: zero critical vulnerabilities.
- Load test: 10,000 concurrent students watching video; p95 latency <300ms.
- DR drill: simulate RDS failure → failover to read replica → recovery in <15 min.

Phase 7: Scale & Advanced Features (Sprints 24+)

Goal: Differentiators that drive retention and revenue.

Feature	Description	Complexity
AI-Powered Recommendations	Collaborative filtering for “Because you enrolled in X”	L
Mobile Apps	React Native or Flutter wrappers for student/admin	XL
Advanced Coaching	Calendar sync (Google/Outlook); payment per session	L
Gamification	Badges, streaks, leaderboards, certificates with blockchain verification	L
Multi-tenancy	White-label for corporate clients	XL
Real-time Collaboration	Live note-sharing during classes	M

Appendix A: Story-to-Phase Mapping

User Story	Phase	Service
US-GUEST-001	2	Student Frontend
US-GUEST-002	2	Course Service + Student Frontend
US-GUEST-003	2	Course Service + Student Frontend
US-GUEST-004	2	Student Frontend
US-GUEST-005	5	Service Request Service
US-GUEST-006	1	Auth Service + Student Frontend
US-STU-001	1	Auth Service
US-STU-002	1	Auth Service
US-STU-003	1	Auth Service
US-STU-004	3	Student Frontend + multiple services
US-STU-005	3	Enrollment Service + Student Frontend
US-STU-006	3	Enrollment Service
US-STU-007	3	Payment Service + Student Frontend
US-STU-008	3	Enrollment Service + Student Frontend
US-STU-009	4	Live Class Service + Student Frontend
US-STU-010	7	Coaching (future)
US-STU-011	5	Service Request Service
US-STU-012	3	Review Service
US-STU-013	1	User Service
US-STU-014	4	Notification Service
US-STU-015	3	Course Service + Student Frontend
US-ADM-001	5	Admin Service + Admin Frontend
US-ADM-002	5	Admin Frontend
US-ADM-003	5	Admin Service + Admin Frontend
US-ADM-004	3 + 5	Payment Service + Analytics Service
US-ADM-005	3 + 5	Enrollment Service + Analytics Service
US-ADM-006	2	Course Service + Admin Frontend
US-ADM-007	2	Upload Service + Admin Frontend
US-ADM-008	2	Course Service
US-ADM-009	2	Course Service

Table 37 – continued

User Story	Phase	Service
US-ADM-010	2	Course Service
US-ADM-011	4	Live Class Service + Admin Frontend
US-ADM-012	1	User Service + Admin Frontend
US-ADM-013	1 + 5	User Service + Admin Frontend
US-ADM-014	5	Service Request Service + Admin Frontend
US-ADM-015	5	Admin Service + Admin Frontend
US-NOTIF-001	4	Notification Service + Admin Frontend
US-NOTIF-002	4	Notification Service + Admin Frontend
US-NFR-001	All	All frontends
US-NFR-002	6	DevOps + all services
US-NFR-003	All	All frontends
US-NFR-004	1 + 6	Auth Service + all layers

Appendix B: Team Structure Recommendation

Phase	Backend	Frontend	DevOps	QA	Total
0-1	2 (Java) + 1 (Node)	1 (Admin) + 1 (Student)	1	1	7
2-3	3 (Java) + 2 (Node)	2 (Admin) + 2 (Student)	1	2	12
4-5	4 (Java) + 3 (Node)	2 (Admin) + 2 (Student)	2	2	15
6-7	4 (Java) + 3 (Node)	2 (Admin) + 2 (Student)	2	3 + 1 (Security)	17

Appendix C: Definition of Done (Per Phase)

A phase is **Done** when:

1. All user stories in the phase meet acceptance criteria.
2. Code review approved by 2+ engineers.
3. Unit test coverage ≥ 80% for new logic.
4. Integration tests pass for all affected service boundaries.
5. UI matches Figma/design spec (pixel-perfect where specified).

6. Accessibility audit passes (axe-core, zero critical violations).
7. Performance budgets met (FCP <1.5s, LCP <2.5s, TTI <3.5s).
8. Security scan passes (Snyk/Trivy: zero critical/high vulnerabilities).
9. Documentation updated (API docs, runbooks, user guides).
10. Product Owner accepts the increment in staging.

End of Implementation Phases Document